Runtime Polymorphism (Method Overriding):

Definition
Runtime polymorphism means that the behavior of a method is decided while program is running, based on the object calling it.

How we implement Runtime Polymorphism?
This happens through Method Overriding a child class provides its own version of a method already defined in the parent class.

Example: This code shows runtime polymorphism using method overriding. The sound() method is defined in base class Animal and overridden in Dog and Cat. At runtime, the correct method is called based on obiect's class.

```
class Animal:
    def sound(self):
        return "Some generic sound"


class Dog(Animal):
    def sound(self):
        return "Bark"


class Cat(Animal):
    def sound(self):
        return "Meow"


animals = [Dog(), Cat(), Animal()]
for animal in animals:
    print(animal.sound())
# Output:
# Bark
# Meow
# Some generic sound
```

Here, sound method behaves differently depending on whether object is a Dog, Cat or Animal and this decision happens at runtime. This dynamic nature makes Python particularly powerful for runtime polymorphism.

Short Questions

1. What is runtime polymorphism?
2. How is runtime polymorphism achieved in Python?

Long Questions

1. Differentiate between compile-time and runtime polymorphism.
2. Write a Python program to demonstrate runtime polymorphism using method overriding.